Solution Architecture

Date	01 July 2026
Team ID	xxxxx
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Solution Architecture Diagram:

Provide a high-level visual representation of your solution's architecture. Use the structural layout below as a template to map out your Presentation Layer, Application Layer, and Data Layer. Fill in the specific technologies and components for your project directly into the boxes to create your architectural diagram.

Presentation / Client Layer		
Web Application - HTML/CSS pages (home.html, predict.html, result.html) rendered via Flask/Jinja2 templates		
↓↑		
API Gateway / Load Balancer		
Flask URL routing (Werkzeug) served through a Gunicorn WSGI server		
↓↑		
Auth Service	**Core Logic Service**	**External API**
Not implemented in current scope (no login/session layer)	Data preprocessing & feature scaling; XGBoost classifier predicts flood risk	None currently integrated (planned: real-time meteorological/weather API)
↓↑		
Data / Storage Layer		
Trained model & scaler files (floods.save); CSV flood prediction dataset (rainfall, cloud visibility, seasonal rainfall)		

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the user interface for entering rainfall/weather details and displaying the flood risk prediction result.	HTML, CSS, Flask (Jinja2 templates)
API Gateway	Routes incoming HTTP requests between the client and the core prediction logic; serves the application.	Flask routing, Gunicorn (WSGI server)
Microservices	Preprocesses rainfall/weather input, scales features, and runs the trained ML model to predict flood risk.	Python, XGBoost, scikit-learn, NumPy, Pandas, Joblib/Pickle
Database	Stores the historical rainfall/weather data used to train the model, and the serialized model/scaler files.	CSV flood prediction dataset (Kaggle), local .save/.pkl file storage